

Game Chat Room Service

Complete Backend Engineering Project Specification

Primary goal: Build a production-oriented, real-time communication backend for multiplayer games, with WebSocket networking, room management, presence, message reliability, Redis-backed state, PostgreSQL persistence, Kafka-based event processing, and a Docker-only development environment.

Development constraint: No Java, Maven, PostgreSQL, Redis, Kafka, Node.js, or other project software needs to be installed directly on the host. All application and infrastructure dependencies run inside Docker containers.

1. Project Overview

The Game Chat Room Service is a real-time backend platform designed for multiplayer games. Players can communicate through global, game-room, team, party, and private chats. The system is intentionally designed to evolve from a single Spring Boot service into a distributed, horizontally scalable architecture.

The project is not intended to be a simple CRUD chat application. Its main learning objectives are real-time networking, concurrent connection management, distributed state, message delivery semantics, fault tolerance, scalability, and event-driven backend architecture.

2. Core Objectives

- Build a real-time WebSocket communication backend.
- Understand persistent TCP/WebSocket connections and connection lifecycle management.
- Implement rooms, memberships, presence, typing indicators, and message history.
- Implement message IDs, sequence numbers, acknowledgements, reconnection, and duplicate detection.
- Use Redis for ephemeral and high-speed state.
- Use PostgreSQL for durable relational data.
- Introduce Kafka for asynchronous event processing.
- Support multiple chat server instances.
- Handle server failures and client reconnection.
- Measure latency, throughput, connection count, resource usage, and failure behavior.
- Keep the complete local development environment Docker-only.

3. Functional Requirements

Area	Requirements
Authentication	Register/login, JWT-based authentication, authenticated WebSocket handshake, user identity validation.
Rooms	Create, join, leave, inspect members, room types, membership rules, room lifecycle.
Chat	Send/receive messages, history, pagination, message IDs, timestamps, sequence numbers.
Chat types	Global, game-room, team, party, and private messaging.
Presence	Online, offline, away, in-game; last-seen information; heartbeat-based liveness.
Realtime	WebSocket connection, server push, typing indicators, join/leave events, delivery acknowledgements.
Reliability	Reconnect, missed-message recovery, duplicate detection, ordering, idempotency.
Safety	Rate limiting, spam controls, mute/kick/ban, reporting, moderation pipeline.

Area	Requirements
Persistence	Users, rooms, memberships, messages, moderation records, audit information.
Scalability	Multiple chat nodes, load balancing, distributed message propagation.
Events	Kafka events for persistence, moderation, analytics, notifications, and auditing.

4. Recommended Technology Stack

Layer	Technology	Purpose
Language	Java 21+	Backend implementation
Framework	Spring Boot 3.x	REST APIs, WebSocket, security, dependency injection
Realtime	Spring WebSocket	Persistent bidirectional client-server communication
Security	Spring Security + JWT	Authentication and authorization
Database	PostgreSQL	Durable relational data
Cache/state	Redis	Presence, active rooms, sessions, rate limits, pub/sub or transient state
Messaging	Apache Kafka	Asynchronous event-driven processing
Containerization	Docker	All application and infrastructure runtime
Orchestration	Docker Compose	Local multi-container development
Testing	JUnit, Mockito, Testcontainers	Unit, integration, infrastructure tests
Load testing	k6 or Gatling	Concurrent users and message throughput testing
Observability	Spring Actuator + logs; later Prometheus, Grafana	Health, metrics, performance monitoring

5. Docker-Only Environment

The host machine should only need Docker Desktop (or a Docker Engine + Compose environment). Java, Maven, PostgreSQL, Redis, Kafka, and other dependencies run inside containers.

```
project/  
  backend/  
    Dockerfile  
    pom.xml  
    src/  
  docker-compose.yml  
  .env.example  
  README.md
```

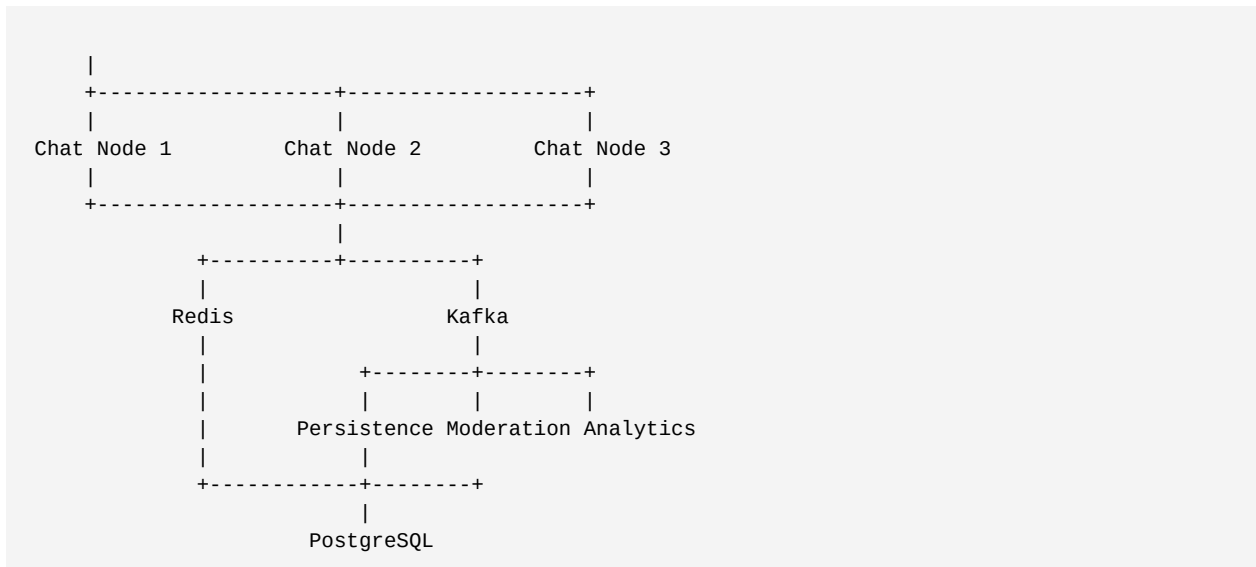
Target container set:

- chat-service: Spring Boot application.
- postgres: durable relational database.
- redis: cache, presence, rate limiting, and transient room state.
- kafka: asynchronous event broker.
- zookeeper or Kafka's KRaft mode: Kafka metadata management depending on chosen Kafka image/configuration.
- optional kafka-ui: browser-based Kafka inspection.
- optional pgadmin: browser-based PostgreSQL administration.
- optional redis-insight: browser-based Redis inspection.
- optional prometheus/grafana in later versions.

The application should compile and run inside its Docker build stage. A multi-stage Dockerfile can use a Maven builder image to compile the application and a lightweight Java runtime image to run it. This avoids installing Maven or Java on the host.

6. High-Level Architecture

```
Game Clients  
  |  
  | HTTPS / WebSocket  
  v  
Load Balancer
```



7. Service Responsibilities

- **Chat Service:** WebSocket connections, message validation, room broadcasting, delivery acknowledgements.
- **Room Service:** Room creation, membership, room policies, team/party/game-room relationships.
- **User/Auth:** Identity, JWT authentication, authorization, account state.
- **Presence:** Connection state, heartbeat, last-seen, online/in-game status.
- **Redis:** Fast ephemeral state and cross-node coordination.
- **Kafka:** Decouples real-time path from persistence, moderation, analytics, and notification consumers.
- **PostgreSQL:** Source of truth for durable entities and message history.

8. REST API Requirements

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Authenticate and receive JWT
POST	/api/rooms	Create room
GET	/api/rooms/{roomId}	Get room information
POST	/api/rooms/{roomId}/join	Join room
POST	/api/rooms/{roomId}/leave	Leave room
GET	/api/rooms/{roomId}/members	List members
GET	/api/rooms/{roomId}/messages	Paginated message history
POST	/api/users/{userId}/mute	Mute user where authorized
POST	/api/reports	Report message/user

9. WebSocket Protocol

Use a structured JSON message envelope. Each event should have a type, request/correlation identifier where useful, and payload.

```

Client -> Server
{
  "type": "SEND_MESSAGE",
  "requestId": "req-123",
  "roomId": "room-42",

```

```

    "content": "Enemy approaching"
  }

  Server -> Client
  {
    "type": "MESSAGE",
    "messageId": "msg-9001",
    "roomId": "room-42",
    "senderId": "user-17",
    "content": "Enemy approaching",
    "sequenceNumber": 18291,
    "timestamp": "2026-08-15T18:30:10Z"
  }

```

Core event types: CONNECTED, JOIN_ROOM, LEAVE_ROOM, SEND_MESSAGE, MESSAGE, MESSAGE_ACK, TYPING_START, TYPING_STOP, USER_ONLINE, USER_OFFLINE, HEARTBEAT, ERROR, SYNC_REQUEST, SYNC_RESPONSE.

10. Database Design

```

users
-----
id (PK)
username
email
password_hash
status
created_at
updated_at

rooms
-----
id (PK)
name
type
owner_id
max_members
created_at
updated_at

room_members
-----
room_id (PK/FK)
user_id (PK/FK)
role
joined_at

messages
-----
id (PK)
room_id (FK)
sender_id (FK)
content
sequence_number
created_at
deleted_at

reports
-----
id (PK)
reporter_id
target_user_id
message_id
reason
status
created_at

```

11. Redis Data Model

presence:user:{id}	-> ONLINE / OFFLINE / IN_GAME
session:user:{id}	-> connection metadata
room:{roomId}:members	-> active member IDs
room:{roomId}:connections	-> active connection mapping
room:{roomId}:sequence	-> latest room sequence
rate:user:{id}:chat	-> rate-limit state
recent:room:{roomId}:messages	-> optional recent-message cache

Use TTLs for ephemeral state so stale connections do not remain forever.

12. Message Reliability and Networking

- Every message receives a globally unique messageId.
- Each room maintains a monotonically increasing sequenceNumber.
- The client sends an acknowledgement after accepting a message.
- The server tracks the last known client sequence where needed.
- On reconnect, the client sends its last received sequence number.
- The server returns messages after that sequence from durable storage/cache.
- Clients deduplicate messages by messageId.
- Use at-least-once semantics for important chat messages; do not claim true exactly-once delivery.
- Use heartbeats to detect dead connections.
- Use exponential backoff on reconnect to avoid reconnect storms.

13. Presence and Connection Lifecycle

```
CONNECT
-> authenticate
-> register connection
-> set presence ONLINE
-> join requested rooms
-> send CONNECTED

HEARTBEAT
-> client/server heartbeat
-> refresh Redis TTL

DISCONNECT
-> remove connection
-> update presence
-> publish USER_OFFLINE

RECONNECT
-> authenticate again
-> restore session/room membership
-> request missed messages
-> resume realtime communication
```

14. Security Requirements

- Passwords must be hashed using a modern password hashing mechanism; never store plaintext passwords.
- JWT access tokens must be validated before WebSocket session authorization.
- Users can only access rooms they are authorized to access.
- Validate and limit message length.
- Reject malformed JSON and unsupported event types.
- Rate-limit message sending and room actions.
- Protect administrative actions such as mute, kick, ban, and moderation.
- Do not trust client-provided sender IDs; derive identity from the authenticated session.
- Use HTTPS/WSS in production.
- Keep secrets in environment variables or a secret manager; do not commit credentials.

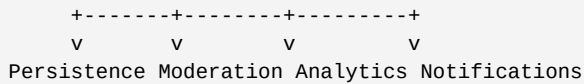
15. Rate Limiting

Implement a Redis-backed token bucket or sliding-window limiter. Example starting policy: 5 chat messages/second/user, with a small burst allowance. Add separate limits for room creation, joining, private messaging, and authentication attempts.

The exact limits should be configurable through environment variables rather than hard-coded.

16. Kafka Event Architecture

```
Chat Service
|
+--> chat.message.sent
+--> chat.message.deleted
+--> chat.user.joined
+--> chat.user.left
+--> chat.user.banned
|
v
Kafka Cluster
|
```



Kafka consumers must be designed for retries and idempotency. Introduce a dead-letter topic for events that repeatedly fail processing.

17. Development Roadmap

Version	Scope	Primary learning
V1	Auth, rooms, WebSocket chat, PostgreSQL	Spring Boot + realtime basics
V2	Global/team/party/private chat, presence, typing, history	Realtime application design
V3	ACKs, sequence numbers, reconnect, missed messages, deduplication	Networking reliability
V4	Redis, multiple chat nodes, load balancing, distributed propagation	Distributed backend
V5	Kafka, persistence/moderation/analytics consumers	Event-driven architecture
V6	Load testing, observability, failure injection, optimization	Production engineering

18. Docker Compose Target

```

services:
  chat-service:
    build: ./backend
    depends_on:
      - postgres
      - redis
      - kafka

  postgres:
    image: postgres

  redis:
    image: redis

  kafka:
    image: apache/kafka

# optional development tools:
# kafka-ui
# pgadmin
# redis-insight

```

Use Docker Compose health checks and service dependency configuration. Persistent named volumes should be used for PostgreSQL and other stateful infrastructure where appropriate.

19. Project Package Structure

```

com.example.gamechat
├── auth
│   ├── controller
│   ├── service
│   ├── security
│   └── dto
├── chat
│   ├── controller
│   ├── websocket
│   ├── service
│   ├── dto
│   └── event
├── room
│   ├── controller
│   └── service

```



```
■ ■■■ repository
■ ■■■ entity
■■■ presence
■■■ moderation
■■■ messaging
■ ■■■ kafka
■ ■■■ redis
■■■ common
■ ■■■ exception
■ ■■■ config
■ ■■■ util
■■■ GameChatApplication.java
```

20. Testing Requirements

- Unit tests for services, validators, rate-limit logic, sequence generation, and authorization.
- Repository integration tests against PostgreSQL.
- Redis integration tests.
- WebSocket integration tests covering connect, authenticate, join room, send, broadcast, disconnect, and reconnect.
- Kafka integration tests for event publishing and consumer processing.
- Test duplicate messages and out-of-order events.
- Test unauthorized room access.
- Test rate limiting.
- Test database failure and Redis unavailability behavior.
- Use Testcontainers where integration tests require real infrastructure.

21. Performance and Load Testing

- Start with 100 concurrent WebSocket clients.
- Progress to 1,000, then 10,000 concurrent connections.
- Measure messages/second and broadcast latency.
- Track p50, p95, and p99 message delivery latency.
- Measure CPU and memory per chat node.
- Measure Redis operations/second and PostgreSQL throughput.
- Test reconnect storms.
- Test one chat node failing while others continue operating.
- Test high-volume rooms separately from many small rooms.

22. Failure Scenarios

- Client loses network connection.
- Client reconnects after several seconds.
- Redis becomes unavailable.
- PostgreSQL becomes temporarily unavailable.
- Kafka consumer crashes.
- Kafka broker becomes unavailable.

- One chat server crashes.
- A client sends duplicate events.
- A client sends malformed or oversized messages.
- A user attempts to access another user's private room.

23. Non-Functional Requirements

- **Availability:** Later versions should continue serving traffic when an individual chat node fails.
- **Latency:** Aim for low end-to-end chat delivery latency; define an explicit p95 target during load testing.
- **Scalability:** Chat nodes should be horizontally scalable.
- **Consistency:** Durable message history should be consistent even when realtime delivery is retried.
- **Security:** Authentication, authorization, rate limiting, input validation, and secret management are mandatory.
- **Observability:** Logs, health endpoints, metrics, correlation IDs, and error tracking.
- **Maintainability:** Clear domain boundaries, DTOs, service interfaces, migrations, tests, and configuration.
- **Reproducibility:** The entire development environment must be reproducible with Docker Compose.

24. Suggested Implementation Order

1. Create the Spring Boot project and Dockerfile.
2. Create Docker Compose with Spring Boot + PostgreSQL.
3. Implement user registration/login and JWT.
4. Create room and membership entities.
5. Implement room REST APIs.
6. Add WebSocket endpoint and authenticated sessions.
7. Implement send/broadcast message flow.
8. Persist messages in PostgreSQL.
9. Add message history and pagination.
10. Add Redis for presence and active room state.
11. Add heartbeat and disconnect handling.
12. Add sequence numbers and acknowledgements.
13. Implement reconnect and missed-message synchronization.
14. Add Redis-based rate limiting.
15. Add private/team/party chat.
16. Run multiple chat-service containers.
17. Add cross-node message propagation.
18. Introduce Kafka.
19. Separate persistence/moderation/analytics consumers.
20. Add load testing and observability.
21. Document architecture, trade-offs, benchmarks, and failure tests.

25. What You Will Learn

- Spring Boot backend architecture.
- REST API design.
- WebSocket and persistent connection handling.
- TCP/networking concepts at the application layer.
- Concurrent connection management.
- Redis caching and ephemeral state.
- PostgreSQL data modeling and transactions.
- Kafka and event-driven architecture.
- Message ordering and delivery semantics.
- Idempotency and duplicate handling.
- Distributed system failure modes.
- Horizontal scaling.
- Docker and containerized development.

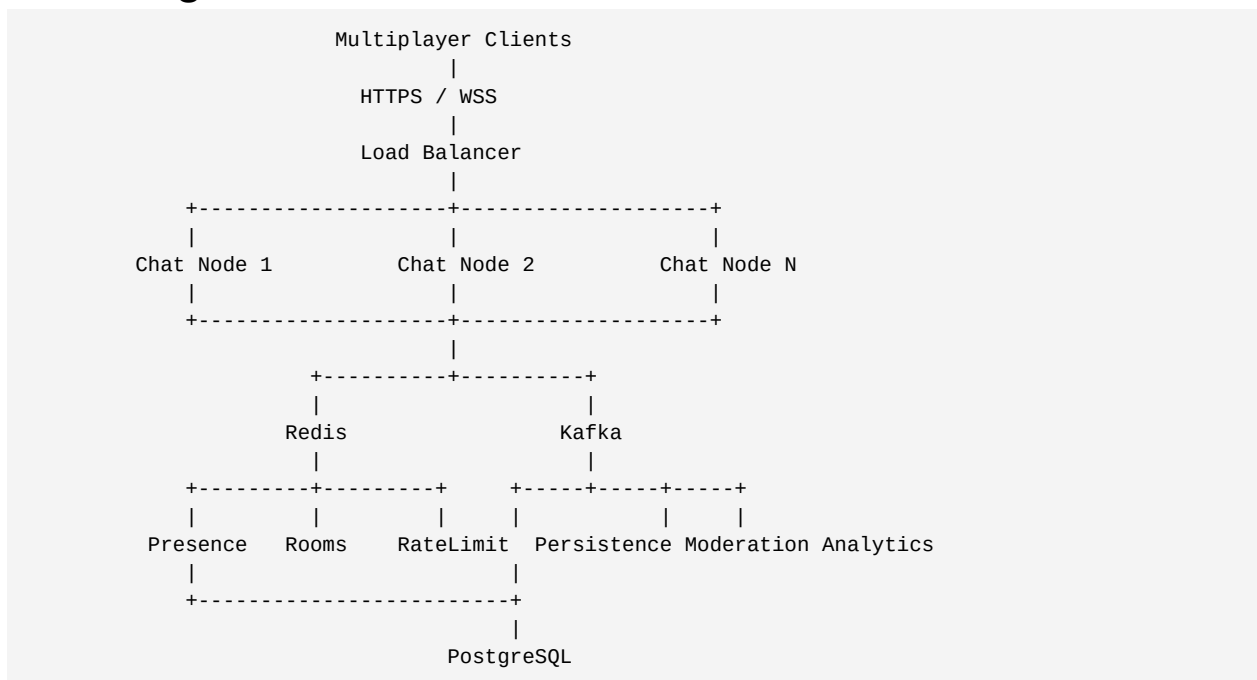
- Load testing and performance engineering.
- Production-oriented observability and security.

26. Portfolio Description

Game Chat Room Service — Distributed Real-Time Communication Backend

Designed and implemented a Dockerized, Spring Boot-based real-time communication platform for multiplayer games using WebSockets, PostgreSQL, Redis, and Kafka. Implemented room-based messaging, presence, connection lifecycle management, message ordering, acknowledgements, reconnect synchronization, rate limiting, and horizontally scalable chat nodes. Added event-driven persistence/moderation pipelines and load-tested concurrent WebSocket connections.

27. Final Target Architecture



28. Definition of Done for the Full Project

- A user can authenticate and establish an authenticated WebSocket connection.
- Users can create and join game chat rooms.
- Messages are delivered to all authorized active members in real time.
- Messages are persisted and retrievable with pagination.
- Presence and heartbeat work correctly.
- Users can reconnect and recover missed messages.
- Messages have IDs and sequence numbers and duplicates are safely handled.
- Chat is rate-limited.
- Multiple chat server containers can run simultaneously.
- Messages can propagate across chat nodes.
- Kafka consumers handle persistence/moderation/analytics events.
- Failures of individual nodes are tested.
- The environment can be started using Docker Compose without installing project dependencies on the host.

- Load tests and performance measurements are documented.
- Architecture, APIs, schema, configuration, and operational instructions are documented.

Recommended first milestone: Build V1 only: Docker Compose + Spring Boot + PostgreSQL + JWT + WebSocket + room management + message persistence. Do not introduce Redis or Kafka until the single-node realtime flow is correct and tested.